

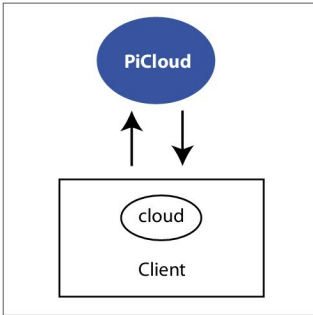


Figure 1: PiCloud components

alternative options, please see <http://docs.picloud.com/moduledoc.html#module-cloud>.

That's all we need to start running code on PiCloud. Let us start with something really simple.

## Running simple functions in the cloud

Now, let's write a simple function and run it on PiCloud. Let us first define a function `sort_num` in the file `demo.py`, as follows:

```
import numpy

def sort_num(num):
    sort_num = numpy.sort(num)
    return sort_num
```

Fire up the Python interpreter and import the function just defined, and the modules `cloud` and `numpy`:

```
>>> from demo import *
>>> import cloud
>>> import numpy
```

Next, let us create an array of 50,000 integers, with each integer picked randomly from the range (10, 10000):

```
>>> num=numpy.random.random_integers(10,10000,50000)
```

Now comes the important step: invoking the `cloud.call` method to specify the function that you want to run, along with its arguments:

```
>>> jid=cloud.call(sort_num, num)
```

This implies that you want the function `sort_num` to be executed with the argument `num`. This method returns an integer, which is an identifier for this particular *job*. The formal specification of the `cloud.call` method is available at <http://docs.picloud.com/moduledoc.html#cloud.call>.

```
>>> jid
58
```

The returned value of the executed function can be obtained using the `cloud.result` method, passing the obtained *jid*:

```
>>> cloud.result(jid)
array([ 10, 11, 11, ..., 10000, 10000, 10000])
```

As you can see, the returned array is obtained.

Now, let's assume that you want to sort 100 similar arrays. You can do this by invoking 100 `cloud.call` methods or you could use a more efficient mechanism—the `cloud.map` method.

## Mapping

Consider a function that returns the volume of a *cylinder* when the *radius*(*r*) and *height*(*h*) is passed to it:

```
import numpy

def vol_cylinder(r, h):
    return numpy.pi*r*r*h
```

If you had one cylinder configuration, i.e., one pair of (*r*, *h*) then the `cloud.call` method could be used for the purpose. What if you had 100 different configurations to find the volume of? You could use 100 `cloud.call` invocations, but there is a more efficient way of doing this—use the `cloud.map` function ([http://docs.picloud.com/client\\_basic.html#mapping](http://docs.picloud.com/client_basic.html#mapping)), which accepts a sequence(s) of parameter(s):

```
>>> from mapdemo import *
>>> import cloud
>>> import numpy
>>> r=numpy.random.random_sample(100)
>>> h=numpy.random.random_sample(100)
>>> jids=cloud.map(vol_cylinder, r, h)
>>> jids
xrange(359, 459)
```

As you can see, *jids* is a list of 100 elements with values from 359 to 459 (note that the values you get